

Performance Analysis and Best-Model Deployment for Spam Detection

Robin

November 26, 2025

0.1 Introduction

Email spam detection is a fundamental binary classification task. This project implements and analyzes a complete hybrid spam–ham classification system using four classical machine learning algorithms—three from scratch and one from scikit-learn—combined into a majority-voting ensemble.

The system includes:

- Multinomial Naïve Bayes
- Logistic Regression (batch gradient descent, from scratch)
- Perceptron (from scratch)
- Linear SVM (TF–IDF, scikit-learn)
- Majority-vote ensemble

The report describes preprocessing, model formulations, hyperparameter tuning, and placeholders for performance metrics.

0.2 Dataset and Preprocessing

0.3 Dataset

The project uses the publicly available Spam Email Dataset from Kaggle. The original dataset consists of labeled email messages, which were provided in a single CSV file. Since the classification system implemented in this work expects raw text files organized into separate folders, a small preprocessing script was written to convert the Kaggle dataset into the required structure:

- `train/spam/` (spam emails)
- `train/ham/` (legitimate emails)
- `test/` (unseen emails for final prediction)

This conversion step allowed the training pipeline to operate directly on raw email text files, consistent with the design of the tokenization, feature extraction, and model training components used in the system.

0.3.1 Tokenization

A custom tokenizer extracts:

- alphanumeric tokens
- hyphenated words
- emails
- URLs
- dollar amounts
- repeated punctuation

0.3.2 Feature Extraction

Effective feature extraction is crucial for representing textual input in a form that machine learning models can interpret. In this project, two complementary feature representations are used: bag-of-words token counts and TF-IDF (weighted sparse vectors). These representations capture different aspects of textual structure and allow each classifier to leverage its underlying assumptions effectively.

Two feature types are used:

- **Bag-of-Words (Count Vectors).**

The bag-of-words (BoW) model represents each email as a high-dimensional sparse vector where each dimension corresponds to a unique token in the vocabulary. The value in each dimension is simply the count of how many times the corresponding token appears in the document. This representation ignores word order and syntactic structure, but preserves frequency information that is highly predictive in spam detection (e.g., repeated promotional terms, commercial keywords, or suspicious URLs).

Count-based vectors are especially suitable for:

- **Multinomial Naïve Bayes**, which directly models token frequencies through a probabilistic generative process.
- **Logistic Regression**, which learns a linear boundary over raw token counts.
- **Perceptron**, which uses the sparsity and binary relevance of token occurrences to adjust its separating hyperplane.

Despite its simplicity, BoW is remarkably powerful in text classification tasks, particularly when documents are long and dominated by distinctive lexical patterns.

- **TF-IDF (Term Frequency-Inverse Document Frequency).**

For the Support Vector Machine classifier, we employ TF-IDF features, which weight tokens not only by their frequency in a document, but also by how rare or informative they are across the entire corpus. The intuition is that words appearing frequently in a particular document (high term frequency) but rarely across all documents (high inverse document frequency) carry more discriminative value.

Formally, each token w receives a weight:

$$\text{TF-IDF}(w, d) = \text{tf}(w, d) \cdot \log \left(\frac{N}{\text{df}(w)} \right)$$

where $\text{df}(w)$ is the number of documents containing w , and N is the total number of documents. We additionally employ a `min_df = 2` constraint to remove extremely rare tokens that contribute noise and do not improve generalization.

TF-IDF is especially well-suited for:

- **Linear SVM**, which benefits from the normalized, high-variance sparse vectors and relies on meaningful token weights to construct a maximum-margin hyperplane.

By combining BoW for probabilistic and incremental learning models with TF-IDF for margin-based classification, the feature extraction pipeline ensures that each algorithm receives an optimal input representation aligned with its strengths. This hybrid feature strategy significantly improves the overall performance of the ensemble.

0.4 Models

0.4.1 Multinomial Naïve Bayes

The Multinomial Naïve Bayes classifier is a probabilistic model that assumes each document is generated by a particular class (spam or ham), and that words inside a document occur independently conditioned on the class. Despite this strong “naïve” independence assumption, the model performs remarkably well in high-dimensional sparse text domains.

The classifier counts how often each word appears in spam emails and ham emails, and converts these counts into conditional word probabilities using Laplace smoothing. During prediction, the model computes the log-likelihood of a document belonging to each class by summing the log-probabilities of each token weighted by its count. The class with the higher log-posterior score is chosen.

Multinomial Naïve Bayes is extremely efficient, requiring only counting and table lookup. It is particularly effective when documents are long and dominated by word-frequency patterns, making it a standard baseline for spam filtering.

Formulation

$$P(c \mid x) \propto P(c) \prod_w P(w \mid c)^{x_w}$$
$$P(w \mid c) = \frac{\text{count}(w, c) + \alpha}{\sum_{w'} \text{count}(w', c) + \alpha|V|}$$

Best α :

$$\alpha^* = \mathbf{0.1}$$

Algorithm

Algorithm 1 Multinomial Naïve Bayes Training

- 1: Initialize vocab and class counts
 - 2: **for** each document **do**
 - 3: Tokenize and update word counts for class
 - 4: **end for**
 - 5: Compute priors and smoothed likelihoods
-

0.4.2 Logistic Regression

Logistic Regression is a discriminative linear model that directly estimates the probability of an email being spam by fitting a sigmoid function over a weighted combination of input features. Unlike Naïve Bayes, which models the generative distribution of words, Logistic Regression focuses purely on learning a decision boundary that best separates spam from ham.

The model optimizes its parameters using gradient descent, minimizing a regularized cross-entropy loss. Each iteration computes the gradient of the loss with respect to the weight vector and bias term, adjusting them in the direction that reduces classification error.

Because Logistic Regression models the log-odds of the target class, it provides probabilistic outputs and interpretable weights. It is robust, well-understood, and often forms a strong baseline in text classification when combined with bag-of-words features.

In this project, a full implementation of Logistic Regression from scratch is developed, including vocabulary construction, vectorization, sigmoid computation, loss evaluation, and batch gradient descent optimization.

Formulation

$$\hat{y} = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss:

$$\mathcal{L} = -\frac{1}{N} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] + \lambda \|w\|^2$$

Tuned:

$$\text{lr} = \mathbf{0.3}, \quad \text{epochs} = \mathbf{40}, \quad \lambda = \mathbf{0.0}$$

Algorithm

Algorithm 2 Logistic Regression: Batch Gradient Descent

- 1: Initialize $w = 0, b = 0$
 - 2: **for** each epoch **do**
 - 3: Compute predictions and gradients
 - 4: Update w and b
 - 5: **end for**
-

0.4.3 Perceptron

The Perceptron is one of the earliest binary linear classifiers and forms the conceptual foundation for modern neural networks. It attempts to learn a hyperplane that separates spam from ham by iteratively adjusting the weights whenever a misclassification occurs.

At each training step, the model predicts a class label using a simple threshold rule based on the dot product of the weight vector and input features. If the prediction is incorrect, the Perceptron performs an update proportional to the input vector and the classification error.

The Perceptron is guaranteed to converge to a perfect separator if the dataset is linearly separable. Although real-world spam data is not perfectly separable, the algorithm can still perform surprisingly well after multiple passes over the training set. Its simplicity, low computational cost, and direct interpretability make it a useful baseline for text classification tasks.

In this project, the Perceptron is implemented entirely from scratch, with custom tokenization, vectorization, and iterative weight updates.

Formulation

$$\hat{y} = \begin{cases} 1 & \text{if } w^\top x + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Update:

$$w \leftarrow w + (y - \hat{y})x, \quad b \leftarrow b + (y - \hat{y})$$

Best epochs:

$$T^* = 20$$

Algorithm

Algorithm 3 Perceptron Learning Rule

- 1: Initialize w and b
 - 2: **for** epoch **do**
 - 3: **for** each sample **do**
 - 4: Predict
 - 5: Update if wrong
 - 6: **end for**
 - 7: **end for**
-

0.4.4 Linear SVM (TF-IDF)

A Linear Support Vector Machine (SVM) constructs a maximum-margin hyperplane that best separates spam from ham in a high-dimensional feature space. Instead of outputting probabilities, SVM focuses on finding the most robust separating boundary by maximizing the distance between the margin and support vectors (training examples closest to the decision boundary).

The model uses the hinge loss, which penalizes points on the wrong side of the margin. An ℓ_2 regularization term prevents overfitting by discouraging excessively large weights. In text classification, TF-IDF features are commonly paired with SVM because the model is especially effective in sparse, high-dimensional environments.

LinearSVC (used in this project) employs a highly optimized coordinate descent algorithm, making it extremely fast even on large vocabularies. SVMs are known to deliver state-of-the-art performance in traditional NLP tasks, often surpassing simpler models like Naïve Bayes and Perceptron.

Because SVMs excel at handling sparse and high-variance token distributions, they typically achieve the strongest individual performance in spam detection.

Formulation

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y_i(w^\top x_i + b))$$

Best:

$$C^* = 1, \quad \text{loss} = \mathbf{hinge}$$

Algorithm

Algorithm 4 Linear SVM (Hinge Loss SGD)

```
1: for epoch do
2:   for each sample do
3:     if  $y(w^\top x + b) < 1$  then
4:       Apply hinge-loss gradient update
5:     end if
6:   end for
7: end for
```

0.4.5 Ensemble: Majority Voting

The ensemble combines predictions from four heterogeneous classifiers: Naïve Bayes, Logistic Regression, Perceptron, and SVM. Each model captures different structural and statistical properties of text data, and their strengths compensate for each other’s weaknesses.

The ensemble uses a simple majority-vote mechanism: if at least two of the four models vote for “spam”, the final output is labeled as spam. This approach reduces variance, improves robustness to noise, and stabilizes predictions. It is especially beneficial when the individual models have different inductive biases and error profiles.

In practice, the ensemble often surpasses the accuracy of any single classifier, particularly on ambiguous or borderline emails where individual models may disagree.

Decision rule:

$$\hat{y} = \begin{cases} 1 & \text{if } \sum_{k=1}^4 \hat{y}^{(k)} \geq 2 \\ 0 & \text{otherwise} \end{cases}$$

0.5 Hyperparameter Tuning

Hyperparameter tuning plays a critical role in improving the performance and generalization capabilities of machine learning models, particularly in text classification where the feature space is high-dimensional and sparse. In this project, a systematic tuning procedure is applied to all four classifiers in order to identify the best-performing configurations.

We adopt an 80–20 train–validation split on the training data, ensuring the split is stratified to preserve the spam-to-ham ratio. Each model is trained on the training portion and evaluated on the held-out validation portion for every combination of hyperparameters in a predefined grid. The configuration achieving the highest validation accuracy is chosen as the optimal setting.

The goal of tuning is to balance bias and variance, prevent overfitting, control model complexity, and achieve superior predictive performance. Because the four models differ widely in their assumptions and update dynamics, their hyperparameters influence learning in distinct ways:

- **Naïve Bayes:** The smoothing parameter α controls how probability mass is distributed to unseen words. A very small α risks overfitting to rare tokens, while a very large one may oversmooth and obscure meaningful frequency differences. A grid search over multiple α values helps identify the balance between these extremes.
- **Logistic Regression:** Three hyperparameters are tuned: learning rate (which affects convergence speed and stability), number of training epochs (how long the model trains), and ℓ_2 regularization (which penalizes large weights to reduce overfitting). Because our implementation is from scratch, tuning the learning rate and number of epochs is especially important to achieve stable convergence.
- **Perceptron:** The primary hyperparameter is the number of epochs. Since the Perceptron updates weights only on misclassified points, additional epochs allow it to revisit

challenging mistakes. However, excessive epochs may lead to overfitting on noisy or non-separable data.

- **Support Vector Machine (Linear SVM):** The penalty parameter C and hinge-loss variant are tuned. A larger C heavily penalizes misclassifications, causing a harder margin, while a smaller C produces a softer, more robust margin. We tune both C and the loss function (**hinge** vs. **squared_hinge**) to find the ideal balance between margin width and misclassification cost.

Grid search is chosen due to its simplicity, interpretability, and effectiveness on moderate-sized datasets. Although more advanced methods such as randomized search or Bayesian optimization could be applied, grid search is sufficient for this setting and provides clear insights into model behavior under different configurations.

After tuning, each model is retrained on the *entire* training set using its best hyperparameters. These optimally configured models are then passed to the ensemble, which aggregates their predictions. Since each model undergoes tuning with a dedicated validation split, the ensemble benefits from the strengths of individually optimized classifiers.

This rigorous hyperparameter search ensures that the final system avoids underfitting, mitigates overfitting, and achieves robust generalization to unseen test emails.

Train-validation split: 80–20.

0.5.1 Naïve Bayes

$$\alpha^* = 0.1$$

0.5.2 Logistic Regression

$$(\text{lr}^*, T^*, \lambda^*) = (0.3, 40, 0.0)$$

0.5.3 Perceptron

$$T^* = 20$$

0.5.4 SVM

$$C^* = 1, \quad \ell^* = \text{hinge}$$

0.6 Results

0.6.1 Validation Accuracy

Model	Hyperparameters	Val Accuracy
Naïve Bayes	$\alpha = \mathbf{0.1}$	0.9913
Logistic Regression	$(\mathbf{0.3}, \mathbf{40}, \mathbf{0.0})$	0.9555
Perceptron	$T = \mathbf{20}$	0.8429
SVM	$C = \mathbf{1}, \text{loss} = \mathbf{hinge}$	0.9930
Ensemble	—	0.9921

0.6.2 Best Model Selection and Final Prediction Generation

Since the true test labels are not available during development, the models cannot be evaluated on the test set directly. All model comparison and selection are therefore performed strictly using validation accuracy obtained from the 80–20 train–validation split. These validation results serve as the only reliable basis for assessing model performance, as the hidden test set remains inaccessible until final evaluation.

During hyperparameter tuning, each of the four classifiers—Naïve Bayes, Logistic Regression, Perceptron, and Linear SVM—was trained and evaluated on the validation split. Among these, the Linear SVM achieved the highest validation accuracy (**0.9930**), outperforming both the ensemble and the other individual models.

Based on this result, the system adopts a simplified and robust deployment strategy: **all final predictions are generated exclusively using the Linear SVM model**. Instead of dynamically comparing models at prediction time, the SVM is directly selected as the final model due to its superior validation performance.

The prediction pipeline proceeds as follows:

1. Load the trained Linear SVM model from disk.
2. Apply the model to each unseen test instance.
3. Produce a binary label (spam = 1, ham = 0) for every test file.
4. Save these predictions in an output file for submission to the external evaluation system, which will compute the final test accuracy on the private dataset.

0.7 How to Run the Model

This section briefly explains how to train the system and how to generate predictions for unseen emails.

Requirements

Make sure you have:

- Python 3.8 or later,
- the required packages (e.g., `scikit-learn`),
- the dataset arranged as:
 - `train/spam/`
 - `train/ham/`
 - `test/`
- the script `spam_classifier.py` in the project folder.

Training

Training performs hyperparameter tuning and then retrains each model on the full training set using the best parameters found. The trained models are saved in `nb_model_ensemble.pkl`. Run:

```
python3 spam_classifier.py train
```

Prediction

For deployment, the system uses the Linear SVM model, which achieved the best validation accuracy. To classify all emails in the `test/` folder, run:

```
python3 spam_classifier.py predict
```

This generates a file `svm_predictions.txt` containing one label per test email (spam = 1, ham = 0). This file can then be submitted for evaluation on the hidden test set.

0.8 Discussion

- SVM typically performs best due to TF-IDF features.
- NB excels in sparse text settings.
- Logistic regression depends heavily on learning rate and regularization.
- Perceptron benefits from increased epochs but remains sensitive.
- Ensemble improves robustness and smooths out weaknesses.

0.9 Conclusion

This project demonstrates that classical machine learning models, when properly tuned and carefully evaluated, can achieve strong performance on the task of spam detection. Although multiple models were trained and compared, the Linear SVM consistently achieved the highest validation accuracy, outperforming Naïve Bayes, Logistic Regression, the Perceptron, and even the ensemble combination. Based on this observation, the SVM was selected as the final model for generating predictions on the unseen test data.

The overall pipeline—consisting of preprocessing, feature extraction, hyperparameter tuning, and validation-based model selection—proved reliable and effective. While ensemble methods generally offer robustness, in this case the validation results clearly indicated that the tuned SVM provided the strongest generalization. Consequently, the deployed system relies exclusively on the SVM to classify new emails as spam or ham.

These results highlight that, even without deep learning, well-structured classical approaches can still deliver high-quality performance in practical text classification tasks.